CHAPTER 2

PIC ARCHITECTURE & ASSEMBLY LANGUAGE PROGRAMMING

Now examine the instructions in Tables 2-2 and 2-3. The instructions in Table 2-2 operate on both WREG and a file register location and then give you the option of placing the result in WREG or a file register location. The instructions in Table 2-3, however, operate on the file register only and then give you the option of placing the result in WREG or a file register location.

Table 2-2: ALU Instructions Using Both WREG and fileReg

Instruction		
ADDWF	fileReg, d	ADD WREG and fileReg
ADDWFC	fileReg, d	ADD WREG and fileReg with Carry
ANDWF	fileReg, d	AND WREG with fileReg
IORWF	fileReg, d	OR WREG with fileReg
SUBFWB	fileReg, d	Subtract fileReg from WREG with borrow
SUBWF	fileReg, d	Subtract WREG from fileReg
SUBWFB	fileReg, d	Subtract WREG from fileReg with borrow
XORWF	fileReg, d	Exclusive-OR WREG with fileReg

Note: The d bit selects the destination for the operation. If d = w; the result is stored in WREG (d = 0). If d = F; the result is stored in the fileReg (d = 1). The default is F. That means "`ADDWF myfile`" is the same as "`ADDWF myfile,F`."

See Chapter 5 for examples of the instructions in Table 2-2.

Table 2-3: File Register Instructions Using fileReg or WREG as Destination

Instruction		
COMF	fileReg, d	Complement fileReg
DECF	fileReg, d	Decrement fileReg
DECFSZ	fileReg, d	Decrement fileReg and skip if zero
DECFSNZ	fileReg, d	Decrement fileReg and skip if not zero
INCF	fileReg, d	Increment fileReg
INCFSZ	fileReg, d	Increment fileReg and skip if zero
INCSNZ	fileReg, d	Increment fileReg and skip if not zero
MOVF	fileReg, d	Move fileReg
NEGF	fileReg, d	Negative fileReg
RLCF	fileReg, d	Rotate left fileReg through carry
RLNCF	fileReg, d	Rotate left fileReg (No carry)
RRCF	fileReg, d	Rotate right fileReg through carry
RRNCF	fileReg, d	Rotate right fileReg (No carry)
SWAPF	fileReg, d	Swap nibbles in fileReg
BTG	fileReg, d	Bit Toggle fileReg

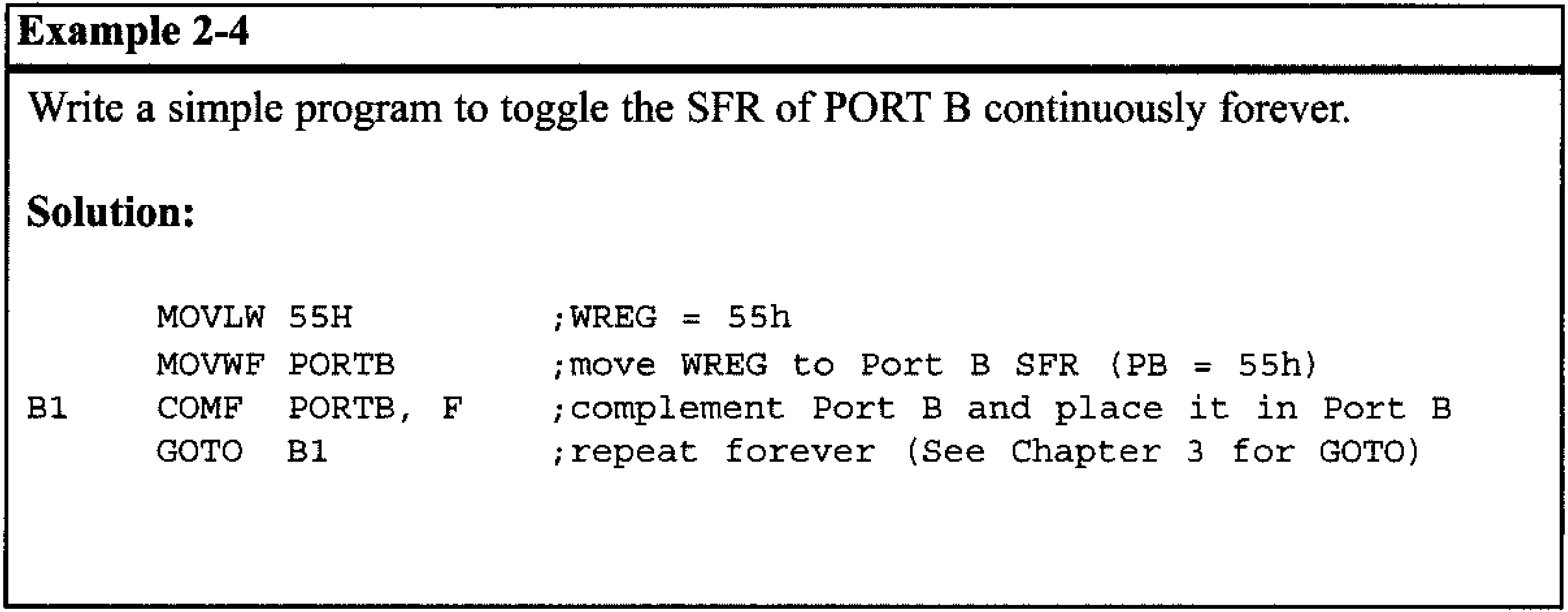
COMF instruction

The "`COMF fileReg, d`" instruction complements (inverts) the contents of fileReg and places the result in WREG or fileReg. This is an example of what is called Read - Modify - Write and we will see more of this in future chapters. In the following program, we put 55H into WREG and then send it to SFR location of Port B. Then the content of Port B is complemented, which becomes AA in hex. The 01010101 (55H) is inverted and becomes 10101010 (AAH).

```
MOVLW 55H          ;WREG = 55h
MOVWF PORTB        ;Move WREG to Port B SFR (PB = 55h)
COMF  PORTB, F     ;complement Port B (PB = AAh)
```

Example 2-4

Write a simple program to toggle the SFR of PORT B continuously forever.



Example 2-4

Write a simple program to toggle the SFR of PORT B continuously forever.

Solution:

```
      MOVLW 55H           ;WREG = 55h
      MOVWF PORTB         ;move WREG to Port B SFR (PB = 55h)
B1    COMF  PORTB, F      ;complement Port B and place it in Port B
      GOTO  B1            ;repeat forever (See Chapter 3 for GOTO)
```

DECF instruction

The “`DECF fileReg, d`” instruction decrements (subtracts one from) the contents of fileReg and places the result in WREG or fileReg. In the following program, we put the value 3 into fileReg location 0x20. Then the value in location 0x20 is decremented and placed in fileReg.

```
MOVLW 3                 ;WREG = 3
MOVWF 20H               ;move WREG to loc 20H (loc 20H = 3)
DECF  0x20, F           ;loc 20H has 2
DECF  0x20, F           ;loc 20H has 1
DECF  0x20, F           ;loc 20H has 0 and WREG = 3
```

Now, contrast the above code with the following:

```
MOVLW 3                 ;WREG = 3
MOVWF 20H               ;move WREG to loc 20H (loc 20H = 3)
DECF  0x20, W           ;loc 20H has 3 (WREG = 2)
DECF  0x20, W           ;loc 20H has 3 (WREG = 2)
DECF  0x20, W           ;loc 20H has 3 (WREG = 2)
```

The above concept will be used in loops in the next chapter.

MOVF instruction

The MOVF mnemonic is intended to perform MOVFW. It has the following format:

```
MOVF         fileReg, D
```

If D = 0, it copies the content of fileReg to WREG. If D = 1, the content of fileReg is copied to itself. While typically we use the MOVF instruction to bring data into WREG from I/O pins, we sometimes use it to copy fileReg to itself for the purpose of testing fileReg contents. Examine the difference between the MOVWF and MOVF instructions. We used the MOVWF instruction earlier to move data to SFRs such as Port B. We also saw how it is widely used to load fixed (literal) data into the RAM locations of the file register because there is no way we can load data into the file register directly. In contrast, the MOVF instruction is widely used to bring data from I/O ports such as Port B into the CPU. We also use the MOVF instruction to bring data into WREG from any SFRs or from any location in the GP RAM in order to perform arithmetic and operations on them. Examine Examples 2-5 and 2-6. Note that the only time we use the "`MOVF fileReg, F`" instruction to copy data from fileReg to itself is when we want to affect the flag bits of the status register. The status register bits are discussed in the next section, and Chapter 3 shows how to use them.

Example 2-5

Write a program to get data from the SFRs of Port B and send it to the SFRs of PORT C continuously.

Example 2-5

Write a program to get data from the SFRs of Port B and send it to the SFRs of PORT C continuously.

Solution:

```
AGAIN MOVF   PORTB, W    ;bring data from PortB into WREG
      MOVWF  PORTC       ;send it to Port C
      GOTO   AGAIN       ;keep doing it forever
```

Example 2-6

Write a program to get data from the SFRs of Port B. Add the value 5 to it and send it to the SFRs of Port C.

Example 2-6

Write a program to get data from the SFRs of Port B. Add the value 5 to it and send it to the SFRs of Port C.

Solution:

```
MOVF  PORTB,W     ;bring data from Port B into WREG
ADDLW 05H         ;add 5 to WREG
MOVWF PORTC       ;copy WREG to Port C
```

MOVFF instruction

The MOVFF instruction copies data from one location in fileReg to another location in fileReg. The fileReg location for source and destination can be any of the 4096 locations in the data RAM space of the PIC18. The MOVFF instruction allows us to move data within the 4K space of the data RAM without going through the WREG register. (See Figure 2-6.) Compare Examples 2-5 and 2-7.

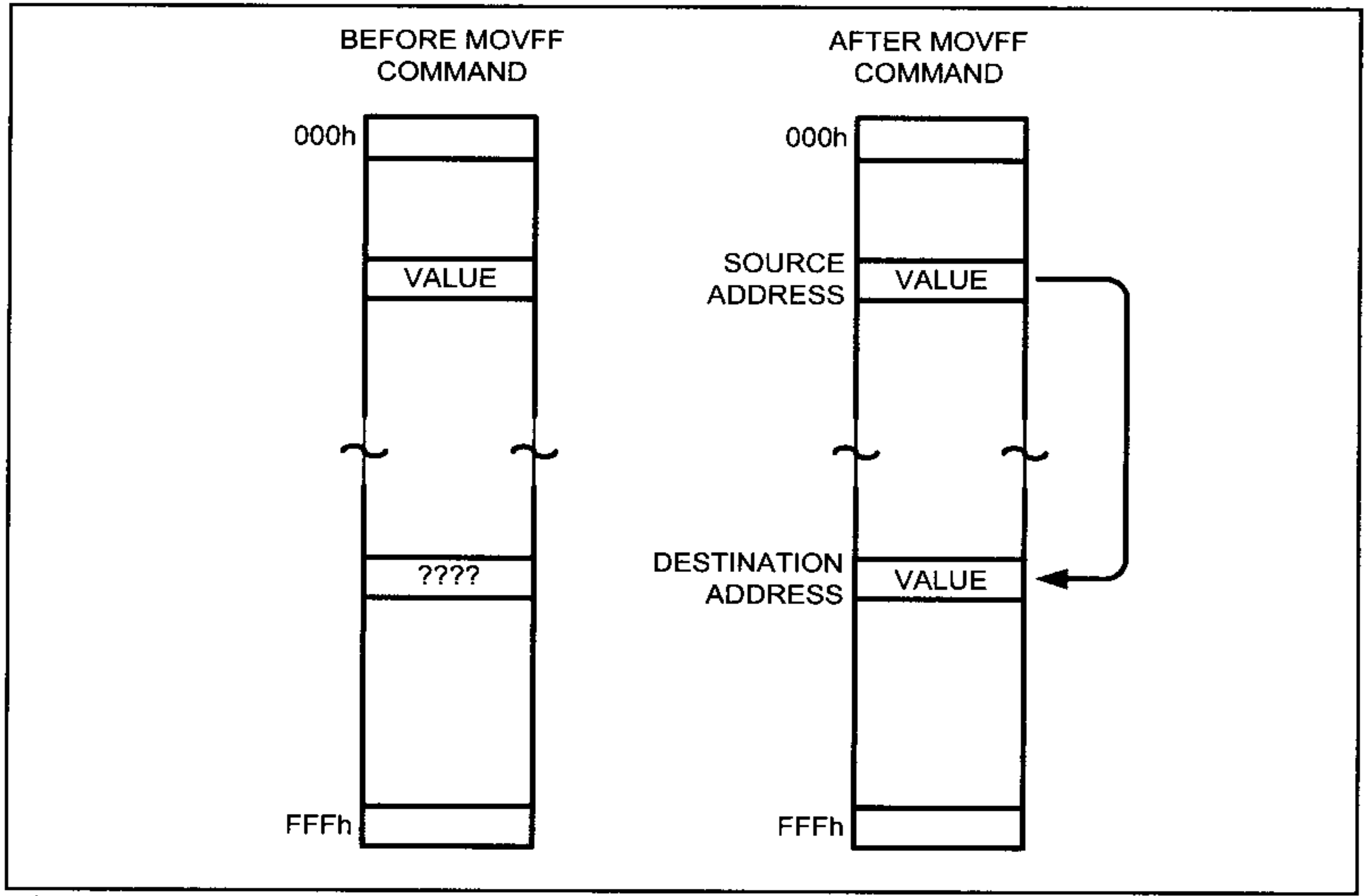

BEFORE MOVFF COMMAND
AFTER MOVFF COMMAND
000h
VALUE
????
FFFh
000h
SOURCE ADDRESS
VALUE
DESTINATION ADDRESS
VALUE
FFFh

Figure 2-6. Moving Data Directly Among the fileReg Locations

Example 2-7

Write a program to get data from the SFRs of Port B and send it to the SFRs of PORT C continuously using MOVFF. Compare this to Example 2-5 and explain the difference.

Example 2-7

Write a program to get data from the SFRs of Port B and send it to the SFRs of PORT C continuously using MOVFF. Compare this to Example 2-5 and explain the difference.

Solution:

```
AGAIN MOVFF PORTB, PORTC        ;copy data from Port B to Port C
      GOTO AGAIN                ;keep doing it forever
```

In Example 2-5 we have:

```
AGAIN MOVF PORTB, W             ;bring data from Port B into WREG
      MOVWF PORTC               ;send it to Port C
      GOTO  AGAIN               ;keep doing it forever
```

Using MOBVFF we simply copy data from one location to another location. But when we use WREG we can perform arithmetic and logic operations on data before it is moved.

SECTION 2.4: PIC STATUS REGISTER

Like all other microprocessors, the PIC has a flag register to indicate arithmetic conditions such as the carry bit. The flag register in the PIC is called the *status register*. In this section, we discuss various bits of this register and provide some examples of how it is altered. Chapters 3 and 5 show how the flag bits of the status register are used.

PIC18 status register

The status register is an 8-bit register. It is also referred to as the *flag register*. Although the status register is 8 bits wide, only 5 bits of it are used by the PIC18. The three unused bits are unimplemented and read as 0. The five flags are called *conditional flags*, meaning that they indicate some conditions that result after an instruction is executed. These five flags are C (carry), DC (digital carry), Z (zero), OV (overflow), and N (negative). See Figure 2-7 for the bits of the status register. Each of the conditional flags can be used to perform a conditional branch (jump), as we will see in Chapter 3.

D7							D0
X	X	X	N	OV	Z	DC	C

C – Carry flag

DC – Digital Carry flag

Z – Zero flag

OV – Overflow flag

N – Negative flag

X – D5, D6, and D7 are not implemented,
and reserved for future use.

Figure 2-7. Bits of Status Register

C, the carry flag

This flag is set whenever there is a carry out from the D7 bit. This flag bit is affected after an 8-bit addition or subtraction. Chapter 5 shows how the carry flag is used.

DC, the digital carry flag

If there is a carry from D3 to D4 during an ADD or SUB operation, this bit is set; otherwise, it is cleared. This flag bit is used by instructions that perform BCD (binary coded decimal) arithmetic. In some microprocessors this is called the AC flag (Auxiliary Carry flag). See Chapter 5 for more information.

Z, the zero flag

The zero flag reflects the result of an arithmetic or logic operation. If the result is zero, then Z = 1. Therefore, Z = 0 if the result is not zero. See Chapter 3 to see how we use the Z flag for looping.

OV, the overflow flag

This flag is set whenever the result of a signed number operation is too large, causing the high-order bit to overflow into the sign bit. In general, the carry flag is used to detect errors in unsigned arithmetic operations while the overflow flag is used to detect errors in signed arithmetic operations. The OV and N flag bits are used for the signed number arithmetic operations and are discussed in Chapter 5.

N, the negative flag

Binary representation of signed numbers uses D7 as the sign bit. The negative flag reflects the result of an arithmetic operation. If the D7 bit of the result is zero, then N = 0 and the result is positive. If the D7 bit is one, then N = 1 and the result is negative. The negative and OV flag bits are used for the signed number arithmetic operations and are discussed in Chapter 5.

Example 2-8

Show the status of the C, DC, and Z flags after the addition of 38H and 2FH in the following instructions:

```
MOVLW 38H
ADDLW 2FH           ;add 2FH to WREG
```

Solution:

38H	0011 1000	
+ 2FH	0010 1111	
67H	0110 0111	WREG = 67H

C = 0 because there is no carry beyond the D7 bit.
DC = 1 because there is a carry from the D3 to the D4 bit.
Z = 0 because the WREG has a value other than 0 after the addition.

Example 2-9

Show the status of the C, DC, and Z flags after the addition of 9CH and 64H in the following instructions:

```
MOVLW 9CH
ADDLW 64H            ;add 64H to WREG
```

Example 2-9

Show the status of the C, DC, and Z flags after the addition of 9CH and 64H in the following instructions:

```
MOVLW  9CH
ADDLW  64H              ;add 64H to WREG
```

Solution:

9CH	1001 1100	
+ 64H	0110 0100	
100H	0000 0000	WREG = 00

C = 1 because there is a carry beyond the D7 bit.
DC = 1 because there is a carry from the D3 to the D4 bit.
Z = 1 because the WREG has a value 0 in it after the addition.

Flag bits and decision making

Because status flags are also called conditional flags, there are instructions that will make a conditional jump (branch) based on the status of the flag bits. Table 2-5 provides the list. Chapter 3 will discuss the conditional branch instructions and how they are used.

Table 2-5: PIC18 Branch (Jump) Instructions Using Flag Bits

Instruction	Action
BC	Branch if C = 1
BNC	Branch if C ≠ 0
BZ	Branch if Z = 1
BNZ	Branch if Z ≠ 0
BN	Branch if N = 1
BNC	Branch if N ≠ 0
BOV	Branch if OV = 1
BNOV	Branch if OV ≠ 0

SECTION 2.5: PIC DATA FORMAT AND DIRECTIVES

In this section we look at some widely used data formats and directives supported by the PIC assembler.

PIC data type

The PIC microcontroller has only one data type. It is 8 bits, and the size of each register is also 8 bits. It is the job of the programmer to break down data larger than 8 bits (00 to FFH, or 0 to 255 in decimal) to be processed by the CPU. For examples of how to process data larger than 8 bits, see Chapter 5. The data types used by the PIC can be positive or negative. A discussion of signed numbers is given in Chapter 5 also. The bit-addressable data is discussed in Chapters 4 and 6.

Data format representation

There are four ways to represent a byte of data in the PIC assembler. The numbers can be in hex, binary, decimal, or ASCII formats. The following are examples of how each work.

Hex numbers

There are four ways to show hex numbers:

1. We can use h (or H) right after the number like this: `MOVLW 99H`
2. Put 0x (or 0X) in front of the number like this: `MOVLW 0x99`
3. Put nothing in front or back of the number like this: `MOVLW 99`
4. Put h in front of the number, but with single quotes around the number like this: `MOVLW h'99'`

We use all four of these methods in this book, because many application notes out there use one of them and we need to get used to them. Notice that some PIC assemblers might give you a warning (but no error) when you use 99H because the assembler already knows that data is in hex and there is no need to remind it. We do that simply to remind ourselves (and it is a good reminder) when we do coding in Assembly.

Here are a few lines of code that use the hex format:

```
MOVLW 25      ;WREG = 25H
ADDLW 0x11    ;WREG = 25H + 11H = 36H
ADDLW 12H     ;WREG = 36H + 12H = 48H
ADDLW H'2A'   ;WREG = 48H + 2AH = 72H
ADDLW 2CH     ;WREG = 72H + 2CH = 9EH
```

The following are invalid:

```
MOVLW E5H     ;invalid, it must be MOVLW 0E5H
ADDLW C6      ;invalid, it must be ADDLW 0C6
```

Notice in the last two instructions that if the value starts with the hex digits A–F, then it must be preceded with a zero. However, the following is valid:

```
MOVLW 0F      ;valid, WREG = 0FH (or 00001111 in binary)
```

Binary numbers

There is only one way to represent binary numbers in a PIC assembler. It is as follows:

```
MOVLW B'10011001' ;WREG = 10011001 or 99 in hex
```

The lowercase b will also work. Note that ' is the single quote key, which is on the same key as the double quote ". This is different from other assemblers such as the 8051 and x86. Here are some examples of how to use it:

```
MOVLW B'00100101' ;WREG = 25H
ADDLW B'00010001' ;WREG = 25H + 11H = 36H
```

Decimal numbers

There are two ways to represent decimal numbers in a PIC assembler. One way is as follows:

```
MOVLW D'12'          ;WREG = 00001100 or 0C in hex
```

The lowercase d will work also. This is different from other assemblers such as the 8051 and x86. In those assemblers, to indicate decimal numbers we simply use the decimal (e.g., 12) and nothing before or after it, while in the PIC assembler, 12 is the default for hex numbers. Here are some examples of how to use it:

```
MOVLW D'37' ;WREG = 25H (37 in decimal is 25 in hex)
ADDLW D'17' ;WREG = 37 + 17 = 54 where 54 in dec is 36H
```

The other way to represent decimal numbers is to use ".value" as seen in some application notes for PIC microcontrollers. This is shown as follows:

```
MOVLW .12            ;WREG = 00001100 = 0CH = 12
```

ASCII character

To represent ASCII data in a PIC assembler we use the letter A as follows:

```
MOVLW A'2'  ;WREG = 00110010 or 32 in hex (See Appendix F)
```

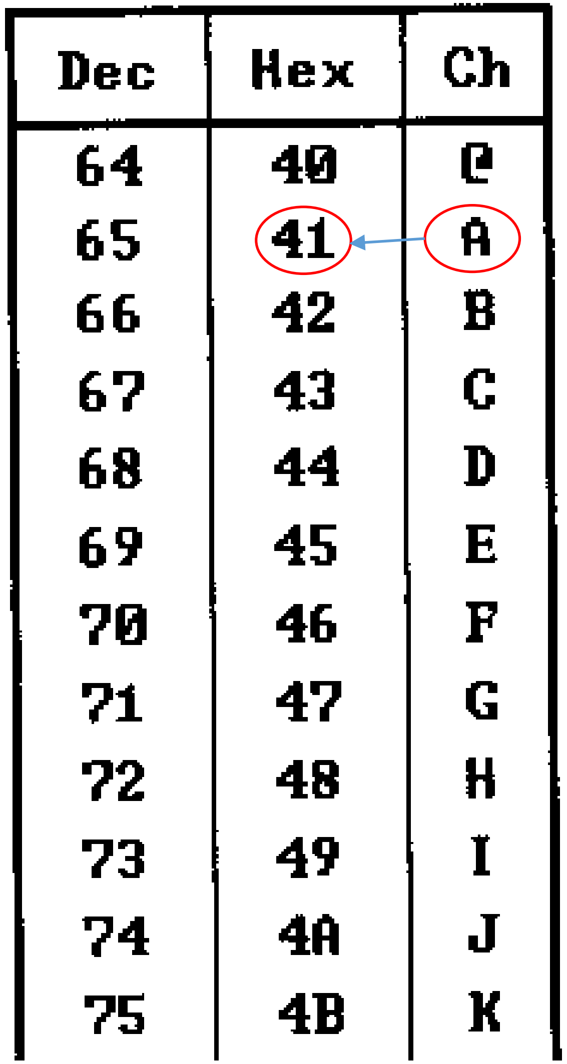
Lowercase ‘a’ will work as well. Again, this is different from other assemblers such as the 8051 and x86. In those assemblers, single quotes are used for single ASCII characters and double quotes are used for a string. Here are some more examples:

```
MOVLW A'9'  ;WREG = 39H, which is hex number for ASCII '9'
ADDLW A'1'  ;WREG = 39H + 31H = 70H
            ;(31 hex is for ASCII '1')
MOVLW '9'   ;WREG = 39H another way for ASCII
```

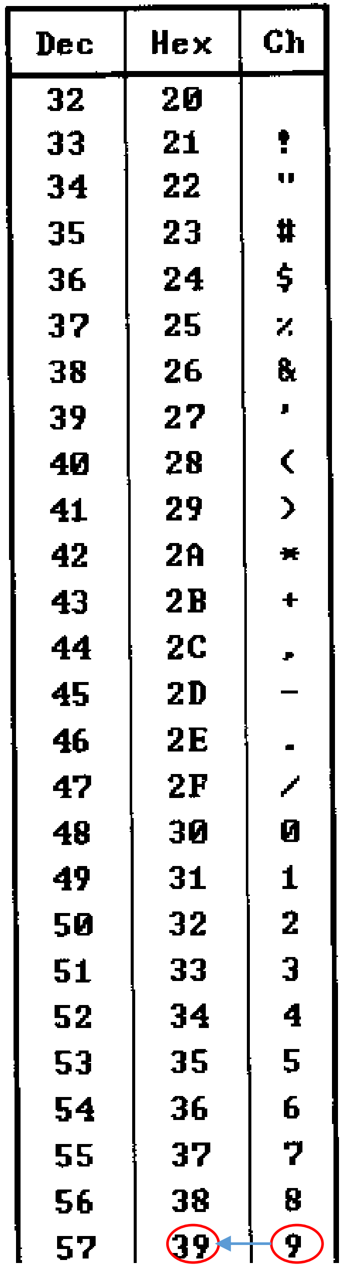
To define ASCII strings (more than one character), we use the DB (define byte) directive. We will see DB usage in Chapter 6.

APPENDIX F

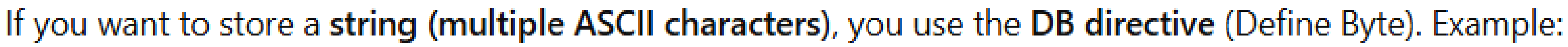
ASCII CODES



Dec	Hex	Ch
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K



Dec	Hex	Ch
32	20	
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9



If you want to store a **string (multiple ASCII characters)**, you use the **DB directive** (Define Byte). Example:

```
MyString DB "HELLO"
```

This stores the ASCII codes of "HELLO" sequentially in memory.

Assembler directives

While instructions tell the CPU what to do, directives (also called pseudo-instructions) give directions to the assembler. For example, the MOVLW and ADDLW instructions are commands to the CPU, but EQU, ORG, and END are directives to the assembler. The following sections present some more widely used directives of the PIC and how they are used.

EQU (equate)

This is used to define a constant value or a fixed address. The EQU directive does not set aside storage for a data item, but associates a constant number with a data or an address label so that when the label appears in the program, its constant will be substituted for the label. The following uses EQU for the counter constant, and then the constant is used to load the WREG register:

```
COUNT EQU   0x25
            ...          ....
      MOVLW        COUNT ;WREG = 25H
```

When executing the above instruction "`MOVLW COUNT`", the register WREG will be loaded with the value 25H. What is the advantage of using EQU? Assume that a constant (a fixed value) is used throughout the program, and the programmer wants to change its value everywhere. By the use of EQU, the programmer can change it once and the assembler will change all of its occurrences throughout the program, rather than search the entire program trying to find every occurrence.

Using EQU for fixed data assignment

To get more practice using EQU to assign fixed data, examine the following:

```
                ;in hexadecimal
DATA1 EQU   39              ;hex data is the default
DATA2 EQU   0x39            ;another way for hex
DATA3 EQU   39H             ;another way for hex (redundant)
DATA4 EQU   H'39'           ;another way for hex
DATA5 EQU   h'39'           ;another way for hex

                ;in binary
DATA6 EQU   b'00110101' ;binary (35 in hex)
DATA7 EQU   B'00110101' ;binary (35 in hex)

                ;in decimal
DATA8 EQU   D'28'           ;decimal numbers (1C in hex)
DATA9 EQU   d'28'           ;second way for decimal

                ;in ASCII
DATA10  EQU  A'2'           ;ASCII characters
DATA11  EQU  a'2'           ;another way for ASCII char
DATA12  EQU  '2'            ;another way for ASCII char
```

ORG (origin)

The ORG directive is used to indicate the beginning of the address. It can be used for both code and data. The number that comes after ORG must be in hex.

END directive

Another important pseudocode is the END directive. This indicates to the assembler the end of the source (asm) file. The END directive is the last line of the PIC program, meaning that anything after the END directive in the source code is ignored by the assembler.

LIST directive

Unlike ORG and END, which are used by all assemblers, the LIST directive is unique to the PIC assembler. It indicates to the assembler the specific PIC chip for which the program should be assembled. It is used as follows:

```
LIST P=18F458
```

The above tells the PIC assembler to assemble the program specifically for the PIC18F458 microcontroller. We use LIST to state the target chip.

#include directive

The #include directive tells the PIC assembler to use the libraries associated with the specific chip for which we are compiling the program.

Rules for labels in Assembly language

By choosing label names that are meaningful, a programmer can make a program much easier to read and maintain. There are several rules that names must follow. First, each label name must be unique. The names used for labels in Assembly language programming consist of alphabetic letters in both upper and lower case, the digits 0 through 9, and the special characters question mark (?), period (.), at (@), underline (_), and dollar sign ($). The first character of the label must be an alphabetic character. In other words, it cannot be a number. Every assembler has some reserved words that must not be used as labels in the program. Foremost among the reserved words are the mnemonics for the instructions. For example, "`MOVWL`" and "ADDWL" are reserved because they are instruction mnemonics. In addition to the mnemonics there are some other reserved words. Check your assembler for the list of reserved words.

SECTION 2.6: INTRODUCTION TO PIC ASSEMBLY PROGRAMMING

In this section we discuss Assembly language format and define some widely used terminology associated with Assembly language programming.

While the CPU can work only in binary, it can do so at a very high speed. It is quite tedious and slow for humans, however, to deal with 0s and 1s in order to program the computer. A program that consists of 0s and 1s is called *machine language.* In the early days of the computer, programmers coded programs in machine language. Although the hexadecimal system was used as a more efficient way to represent binary numbers, the process of working in machine code was still cumbersome for humans. Eventually, Assembly languages were developed, which provided mnemonics for the machine code instructions, plus other features that made programming faster and less prone to error. The term *mnemonic* is frequently used in computer science and engineering literature to refer to codes and abbreviations that are relatively easy to remember. Assembly language programs must be translated into machine code by a program called an *assembler*. Assembly language is referred to as a *low-level language* because it deals directly with the internal structure of the CPU. To program in Assembly language, the programmer must know all the registers of the CPU and the size of each, as well as other details.

Today, one can use many different programming languages, such as BASIC, Pascal, C, C++, Java, and numerous others. These languages are called *high-level languages* because the programmer does not have to be concerned with the internal details of the CPU. Whereas an *assembler* is used to translate an Assembly language program into machine code (sometimes also called *object code* or opcode for operation code), high-level languages are translated into machine code by a program called a *compiler*. For instance, to write a program in C, one must use a C compiler to translate the program into machine language. Next we look at PIC Assembly language format.

Structure of Assembly language

An Assembly language program consists of, among other things, a series of lines of Assembly language instructions. An Assembly language instruction consists of a mnemonic, optionally followed by one or two operands. The operands are the data items being manipulated, and the mnemonics are the commands to the CPU, telling it what to do with those items.

An Assembly language program (see Program 2-1) is a series of statements, or lines, which are either Assembly language instructions such as ADDLW and MOVWF, or statements called directives. While instructions tell the CPU what to do, directives (also called pseudo-instructions) give directions to the assembler. For example, in Program 2-1 while the MOVWF and ADDLW instructions are commands to the CPU, ORG and END are directives to the assembler. The directive ORG tells the assembler to place the opcode at memory location 0 while END indicates the end of the source code to the assembler. In other words, one directive is for the start of the program and the other for the end of the program.

```
      ;PIC Assembly Language Program To Add Some Data.
      ;store SUM in fileReg location 10H.

SUM   EQU   10H         ;RAM loc 10H for SUM

      ORG   0H          ;start at address 0
      MOVLW 25H         ;WREG = 25
      ADDLW 0x34        ;add 34H to WREG
      ADDLW 11H         ;add 11H to WREG
      ADDLW D'18'       ;W = W + 12H = 7CH
      ADDLW 1CH         ;W = W + 1CH = 98H
      ADDLW B'00000110' ;W = W + 6 = 9EH
      MOVWF SUM         ;save the SUM in loc 10H
HERE  GOTO HERE         ;stay here forever
      END               ;end of asm source file
```

Program 2-1: Sample of an Assembly Language Program

Brackets indicate that a field is optional and not all lines have them. Brackets should not be typed in. Regarding the above format, the following points should be noted:

1. The label field allows the program to refer to a line of code by name. The label field cannot exceed a certain number of characters. Check your assembler for the rule.
2. The Assembly language mnemonic (instruction) and operand(s) fields together perform the real work of the program and accomplish the tasks for which the program was written. In Assembly language statements such as

```
MOVLW 55H
ADDLW 67H
```

ADDLW and MOVLW are the mnemonics that produce opcodes; the "55H" and "67H" are the operands. Instead of a mnemonic and an operand, these two fields could contain assembler pseudo-instructions, or directives. Remember that directives do not generate any machine code (opcode) and are used only by the assembler, as opposed to instructions that are translated into machine code (opcode) for the CPU to execute. In Program 2-1 the commands ORG (origin) and END are examples of directives. More of these pseudo-instructions are discussed in future chapters.

3. The comment field begins with a semicolon comment indicator ";". Comments may be at the end of a line or on a line by themselves. The assembler ignores comments, but they are indispensable to programmers. Although comments are optional, it is recommended that they be used to describe the program in a way that makes it easier for someone else to read and understand.
4. Notice the label "HERE" in the label field in Program 2-1. In the GOTO the PIC is told to stay in this loop indefinitely. If your system has a monitor program you do not need this line and it should be deleted from your program. In

```
      ;PIC Assembly Language Program To Add Some Data.
      ;store SUM in fileReg location 10H.

SUM   EQU   10H         ;RAM loc 10H for SUM

      ORG   0H          ;start at address 0
      MOVLW 25H         ;WREG = 25
      ADDLW 0x34        ;add 34H to WREG
      ADDLW 11H         ;add 11H to WREG
      ADDLW D'18'       ;W = W + 12H = 7CH
      ADDLW 1CH         ;W = W + 1CH = 98H
      ADDLW B'00000110' ;W = W + 6 = 9EH
      MOVWF SUM         ;save the SUM in loc 10H
HERE  GOTO HERE         ;stay here forever
      END               ;end of asm source file
```

Program 2-1: Sample of an Assembly Language Program

Example 2-11

Find the ROM memory address of each of the following PIC chips:

(a) PIC18F2220 with 4 KB
(b) PIC18F2410 with 16 KB
(c) PIC18F458 with 32 KB

Solution:

Example 2-11

Find the ROM memory address of each of the following PIC chips:
(a) PIC18F2220 with 4 KB
(b) PIC18F2410 with 16 KB
(c) PIC18F458 with 32 KB

Solution:

(a) With 4K of on-chip ROM memory space, we have 4096 bytes (4 × 1024 = 4096). This maps to address locations of 0000 to 0FFFH. Notice that 0 is always the first location.

(b) With 16K of on-chip ROM memory space, we have 16,384 bytes (16 × 1024 = 16,384),which gives 0000–3FFFH.

(c) With 32K we have 32,768 bytes (32 × 1024 = 32,768). Converting 32,768 to hex, we get 8000H; therefore, the memory space is 0000 to 7FFFH.